AGSTU	TEKNISK RAPPORT	AGSTU
Författare: Darwin Rengifo E-postadress :	Uppgift: TEIS ECS – Embedded Computer System Fil: darwin_rengifo_vhdl2_uppgift_1a.pdf Version : V1.0	

TEIS ECS - Embedded Computer System -

Darwin Rengifo

2018-10-09

Sammanfattning:
Det är en prototyp av VGA-kontroller som ska konstruerar med en toppnivå file för instansierar komponenten, en klock-komponent som kommer att dela 50Mhz klockan till 25Mhz för VGA synkpulser och räknare, en counter-komponent som kommer att räknar en skärm av 640 x 480 pixel och en RGB-komponent som kommer att visar var röd, grön och blå färg ska ritas ut på skärmen.

Innehåll

1. INLEDNING 3
2. KRAVSPECIFIKATION 3
3. BESKRIVNINGEN VGA INTERFACET PÅ KORTET 4
4. BESKRIVNINGEN VGA PROTOKOLLET. 5
5. TESTPROTOKOLL FÖR VERIFIERING OCH ETT ENKELT PROTOKOLL FÖR VALIDERING 6
   5.1 TESTPROTOKOLL FÖR VERIFIERING 6
   5.2 ENKELT PROTOKOLL FÖR VALIDERING 7
6. KONSTRUKTIONSBESKRIVNING 7
   6.1 SYSTEM ARKITEKTUR 7
   6.2 DELSYSTEM 8
   6.2.1 TOPPNIVÅN 8
   6.2.1.1 SYSTEM OCH ARKITEKTUR 8
   6.2.2 COUNTER - KOMPONENT 9
   6.2.2.1 SYSTEM OCH ARKITEKTUR 9
   6.2.3 DISPLAY RGB - KOMPONENT 10
   Instansnamn: b2v_inst_display_rgb 10
   6.2.3.1 SYSTEM OCH ARKITEKTUR 10
   6.2.4 KLOCK - KOMPONENT 11
   Instansnamn: b2v_inst_ klock_delare 11
   6.2.4.1 SYSTEM OCH ARKITEKTUR 11
7. VERIFIERING 12
   7.1 TESTBÄNKEN 12
   7.2 BILDER PÅ RESULTATEN FRÅN SIMULERINGEN 12
   7.3 IFYLLT TESTPROTOKOLL FÖR VERIFIERING 13
8. VALIDERING 14
9. BILAGA 15

1. INLEDNING

Denna rapport beskriver som bygga en VGA-prototyp som styrs av några test knappar och där resultatet visas på en VGA-skärm (för att validera konstruktionen). Vidare behövs även en VGA-kabel mellan kortet och skärmen. Systemet har analyserats genom simulering och verifiering i ModelSim och därefter validerats på ett lämpligt FPGA-kort.

2. KRAVSPECIFIKATION

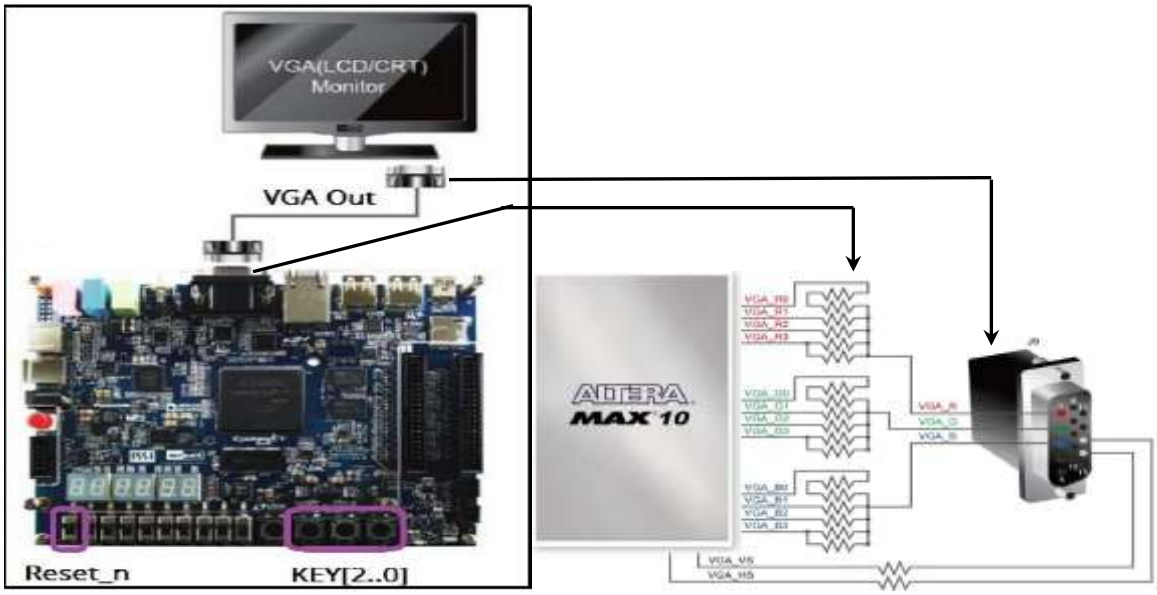
Tabell 1. Kravspecifikation från vhdl2_uppgift_1a

Krav id	Beskrivning	Utfört Ja/nej
	Konstruktionskrav	
1	Skapa ett Quartus-projekt med namnet vhdl2_uppgift_1a	Ja
2a	Kravspecifikationen är att konstruera en VGA-kontroller, kommentera koden, verifiera med testbänk (före valideringen) och validera koden på DE10-kortet.	Ja
2b	VGA-protokollet ska följas och dela 50 MHz klockan i en process för att få 25 MHz.	Ja
2c	Antal bitar för färgerna är fritt att välja. Frivilligt att göras generisk.	
	Testprotokoll	
3	Testfall: VGA protokollets styrsignaler testas med ModelSim. Följande signaler ska verifieras: VGA_HS och VGA_VS. Se bilaga. Validering på FPGA kortet: Testfall 1: Rita en röd fyrkant i övre vänster hörn när KEY(0) trycks ned Testfall 2: Rita en grön fyrkant i nedre vänster hörn när KEY(1) trycks ned Testfall 3: Rita en blå fyrkant i nedre höger hörn när KEY(2) trycks ned Dessa färger ska överlappa varandra och de ska blandas i mitten av skärmen. Se figur 14 i bilaga D.	Ja
	VHDL-kod	
4	Regler och riktlinjer för VHDL och C ska följas.	Ja
	Verifierings-/Valideringskrav	
5a	Verifiera med ModelSim.	Ja
5b	Validera på DE1-kortet. Fyll i testprotokollet.	Ja
	Rapportkrav	

6	Skriv en rapport (pdf eller word): 1) Framsida med titel, ditt namn, e-post, datum och en kort sammanfattning 2) Innehållsförteckning, sidnumrering och kapitel nummer 3) Kravspecifikation, eget kapitel. 4) Beskriv VGA interfacet på kortet. 5) Beskriv VGA-protokollet. Ett tydligt pulsdiagram i förhållande till H- och V-räknarna för VGA-signalerna: VGA_HS och VGA_VS. (detta bör göras innan koden skrivs!), eget kapitel. 6) Testprotokoll för verifiering och ett enkelt protokoll för validering (olika protokoll), eget kapitel. 7) Konstruktionsbeskrivning, eget kapitel. a. System arkitektur b. Delsystem 8) Verifiering, Eget kapitel. a. Testbänken, Teststimuli (som bygger på testprotokollet för programmet) b. Bilder på resultatet från simuleringen med ModelSim (se bilaga C för exempel) c. Ifyllt testprotokoll för verifiering 9) Validering, beskriv resultatet med ett foto. a. Ifyllt testprotokoll för validering (ej samma som verifiering) 10) Bilaga a. VHDL-fil b. Do-fil c. Testbänk	Ja
	Ett dokumentationskrav är också att figurer och tabeller ska ha numrering och beskrivning. Figurbeskrivning under figuren och tabellbeskrivning ovanför tabellen.	Ja
Leveranskrav		
7	Leveransen ska ske till Itslearning. Leveransen ska vara rapporten och det arkiverade projektet. Namnet på filen ska vara ”förnamn_efternamn_vhdl2_uppgift_1a”. Sista leveransdag se kursschema.	Ja

3. BESKRIVNINGEN VGA INTERFACET PÅ KORTET.

DE10-Lite-kortet innehåller en 15-pin D-SUB-kontakt för VGA-utgång. Synkronisering signaler kommer rätt från DE10-Lite kortet och använder en 4-bit DAC som använder en resistens nätverk för att producerar data signaler analoga (röd, grön, blå). På Figur 1 visar som är VGA interfacet på kortet.



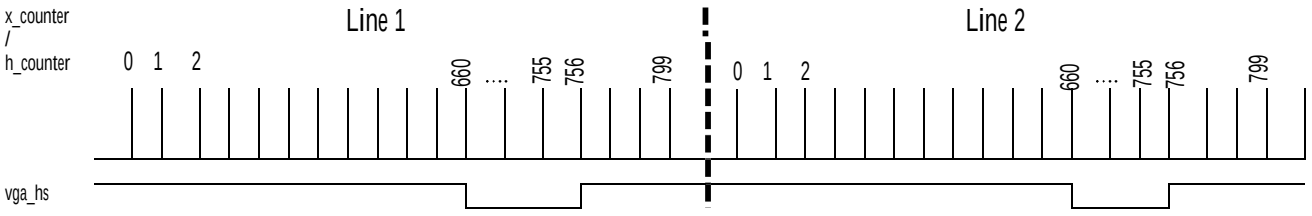
Figur 2. Anslutning mellan FPGA kortet DE10-Lite och VGA skärm

4. BESKRIVNINGEN VGA PROTOKOLLET.

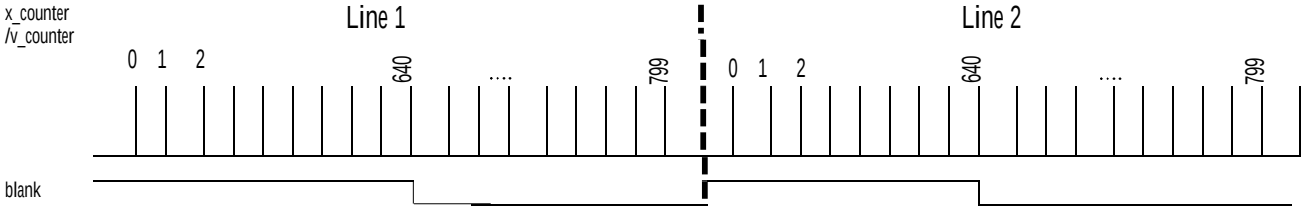
På dessa följande pulsdiagram visar H- och V-räknarna för VGA-signalerna: vga_hs, vga_vs och blank.

Figur 3 visar som horisontell sync(vga_hs) aktiverar sig med fallande flank från horisontell räknaren 660 till 755, som är den tid krävs för att genererar alla pixlar i en hel horisontell linje.
Figur 4 visar som vertikal sync(vga_vs) aktiverar sig med fallande flank på vertikal räknaren 494, som är den tid krävs för att genererar alla linjer i skärmen(ram).

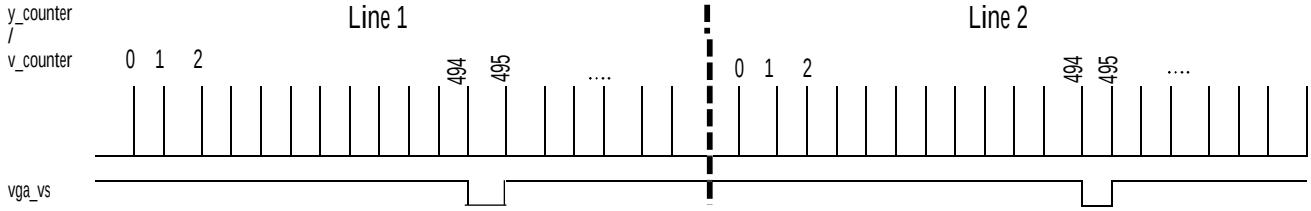
Figur 5 visar som blank(blank) aktiverar sig med fallande flank från horisontell räknaren 640 till 799 och genererar mörkt skärmen för horisontella linjer.



Figur 3. Visar räknaren för horisontell sync



Figur 4. Visar räknaren för blank



Figur 5. Visar räknaren för vertikal sync

5. TESTPROTOKOLL FÖR VERIFIERING OCH ETT ENKELT PROTOKOLL FÖR VALIDERING.

5.1 TESTPROTOKOLL FÖR VERIFIERING

Tabell 2. Testprotokoll för verifiering

Testfall	Beskrivning	Acceptansbeskrivning	Verifiering
1	Gör en reset av systemet. Hela skärmen skall rensas vid uppstart av programmet.	Reset_n = '1'	
2	Horisontell sync(vga_hs) med fallande flank.	Horisontell sync(vga_hs) börjar med fallande flank, från horisontell räknaren 660.	
3	Horisontell sync(vga_hs) med stigande flank.	Horisontell sync(vga_hs) slutar med stigande flank, på horisontell räknaren 755	
4	Blank med fallande flank.	Blank(blank) börjar med fallande flank, från horisontell räknaren 640.	
5	Blank med stigande flank.	Blank(blank) slutar med stigande flank, på horisontell räknaren 799.	
6	Vertikal sync(vga_vs) med fallande flank.	Vertikal sync(vga_vs) börjar med fallande flank, från vertikal räknaren 494.	
7	Vertikal sync(vga_vs) med stigande flank.	Vertikal sync(vga_vs) slutar med stigande flank, på vertikal räknaren 494	
8	Övergång till nästa rad	Visa alla signaler till nästa rad	

5.2 ENKELT PROTOKOLL FÖR VALIDERING

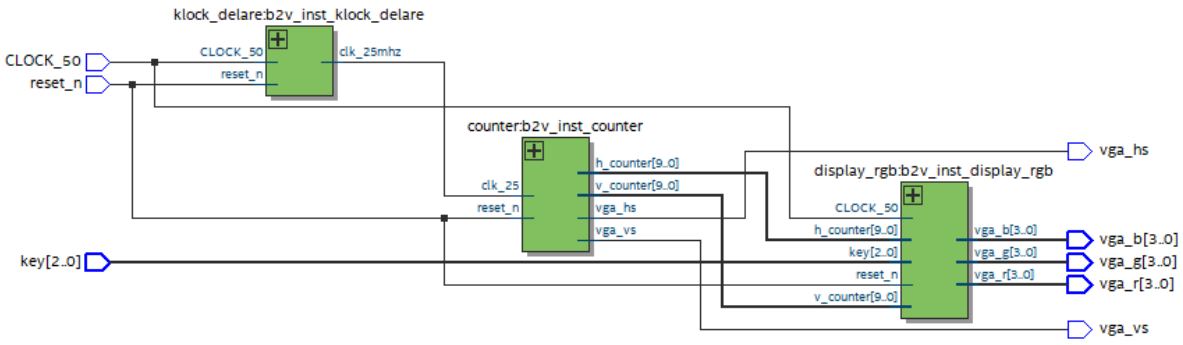
Tabell 3. Testprotokoll för validering

Testfall	Beskrivning	Acceptansbeskrivning	Verifiering
1	När trycks ned Sw(0)	Rita en röd fyrkant i övre vänster hörn	
2	När trycks ned Sw(1)	Rita en grön fyrkant i nedre vänster hörn	
3	När trycks ned Sw(2)	Rita en blå fyrkant i nedre höger hörn	
4	När alla switch är nedre	Dessa färger ska överlappa varandra och de ska blandas i mitten av skärmen	

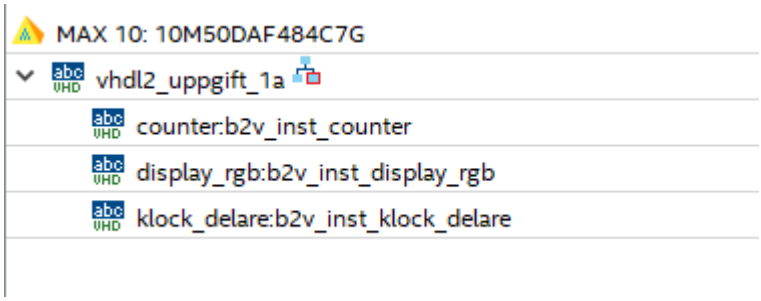
6. KONSTRUKTIONSBESKRIVNING

Programmet körs på kortet DE10-Lite är en robust hårdvara design plattform som byggde kring Altera MAX 10 FPGA med en Nios II-CPU som är anpassad för ingenjörsjobbet. DE10-Lite-kortet innehåller en 15-pin D-SUB-kontakt för VGA-utgång, VGA skärm för att visualisering resultaten, VGA-kabel för att skickar signalerna från DE10-Lite kortet till VGA skärm och tre brytarna för att kan testar VGA-kontroller.

6.1 SYSTEM ARKITEKTUR



Figur 6. Systemarkitekturen för TEIS datorsystem



Figur 7. Komponenthierarki för VGA-Kontroller

6.2 DELSYSTEM

Systemet innehålla följande komponenter:

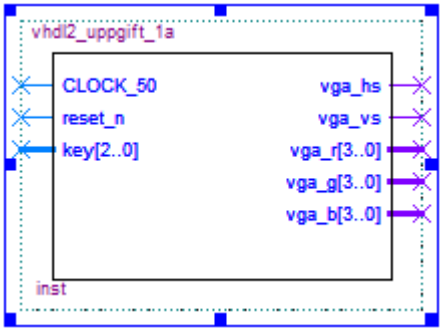
6.2.1 TOPPNIVÅN

Komponent namn: ***vhdl2_uppgift_1a***

6.2.1.1 SYSTEM OCH ARKITEKTUR

Den toppnivån användas för att får ingången signaler, instansierar de andra komponenter och skickar utgången signaler till skärmen.

TOPPNIVÅN symbol visas i Figur 8 och tabell 7 visar In- och utsignalerna.



Figur 8. Toppnivån för VGA-kontroller

Tabell 4. In- och utsignalerna till topp filen

Signal	Namn	Riktning	Typ
Klocksignal 50 MHz	CLOCK_50	in	std_logic
Switch reset	reset_n	in	std_logic
Bytrarna	key	in	std_logic_vector(2 downto 0)
Horizontell synkronism	vga_hs	out	std_logic
Vertikal synkronism	vga_vs	out	std_logic
Röd signal	vga_r	out	std_logic_vector(3 downto 0)
Grön signal	vga_g	out	std_logic_vector(3 downto 0)
Blå signal	vga_b	out	std_logic_vector(3 downto 0)

Ingången signaler

CLOCK_50: 50Mhz input clock.

reset_n: Reset av signalen. Active low.
key: For att rita fyrkanter i vissa skärmen positioner.

Utgången signaler
vga_hs: Horizontell synkronism. Den kontrollerar början och slutet på en rad och ser till att skärmen visar pixlarna mellan vänster och höger del av den synliga delen av bildskärmen.
vga_vs : Vertikal synkronism. Kontrollerar början och slutet på en ram(frame) och ser till att den visas mellan topp och nedre kanten och inom skärmen synliga ram, alla linjer som sminka.
vga_r: Definierar röd färg av signalen.
vga_g: Definierar grön färg av signalen.
vga_b: Definierar blå färg av signalen.

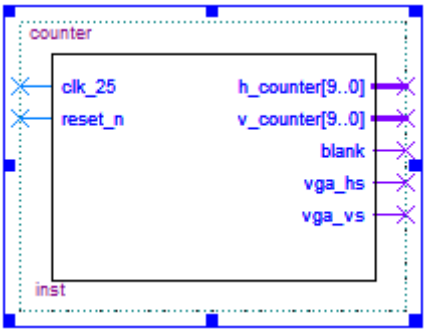
6.2.2 COUNTER - KOMPONENT

Komponent namn: ***counter***
Instansnamn: ***b2v_inst_counter***

6.2.2.1 SYSTEM OCH ARKITEKTUR

Den komponent användas för att räknar en skärm av 640 x 480 pixel för att aktivera horisontell, vertikal och mörk synkronisering.

COUNTER-komponentens symbol visas i Figur 9 och tabell 5 visar In- och utsignalerna.



Figur 9. Counter för VGA-kontroller

Tabell 5. In- och utsignalerna till counter

Signal	Namn	Riktning	Typ
Klocksignal 25 MHz	clk_25	in	std_logic
Switch reset	reset_n	in	std_logic
Horizontell counter	h_counter	out	std_logic_vector(9 downto 0)
Vertikal counter	v_counter	out	std_logic_vector(9 downto 0)
Mörk skärm	blank	out	std_logic
Horizontell syn	vga_hs	out	std_logic

Vertikal sync	vga_vs	out	std_logic

Ingången signaler
clk_25: 25Mhz input clock för VGA synkpulser och räknare.
reset_n: Reset av signalen. Active low.

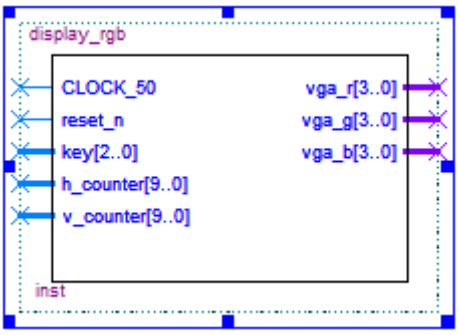
Utgången signaler
h_counter: Det kommer att räkna från 0 till 799, även om den synliga bilden endast är 640 pixlar bred.
v_counter: Det kommer att räkna från 0 till 525, även om den synliga bilden endast är 480 pixlar hög.
blank: Det kommer att aktiverar(”0”) när Horisontellräknare(h_counter) är mellan (640 till 799).
vga_hs: Det kommer att aktiverar(”0”) när h_counter är mellan (660 till 755)
vga_vs: Det kommer att aktiverar('0') när "v_counter= 494"

6.2.3 DISPLAY RGB - KOMPONENT

Komponent namn: ***display_rgb***
Instansnamn: b2v_inst_display_rgb

6.2.3.1 SYSTEM OCH ARKITEKTUR

Den komponent användas för att visar var röd, grön och blå färg ska ritas ut på skärmen enligt testfallen. I mitten ska färgerna blandas med varandra.
DISPLAY_RGB-komponentens symbol visas i Figur 10 och tabell 6 visar In- och utsignalerna.



Figur 10. Display RGB för VGA-kontroller

Tabell 6. In- och utsignalerna till display_rgb

Signal	Namn	Riktning	Typ
Klocksignal 50 MHz	CLOCK_50	in	std_logic
Switch reset	reset_n	in	std_logic
Bytrarna	key	in	std_logic_vector(2 downto 0)
Horizontell counter	h_counter	in	std_logic_vector(9 downto 0)

Vertikal counter	v_counter	in	std_logic_vector(9 downto 0)
Röd signal	vga_r	out	std_logic_vector(3 downto 0)
Grön signal	vga_g	out	std_logic_vector(3 downto 0)
Blå signal	vga_b	out	std_logic_vector(3 downto 0)

Ingången signaler
CLOCK_50: 50Mhz input clock.
reset_n: Reset av signalen. Active low.
key: For att rita fyrkanter i vissa skärmen positioner.

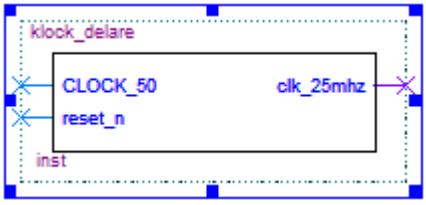
Utgången signaler
h_counter: Det kommer att räkna från 0 till 799, även om den synliga bilden endast är 640 pixlar bred.
v_counter: Det kommer att räkna från 0 till 525, även om den synliga bilden endast är 480 pixlar hög.
vga_r: Definierar röd färg av signalen.
vga_g: Definierar grön färg av signalen.
vga_b: Definierar blå färg av signalen.

6.2.4 KLOCK - KOMPONENT

Komponent namn: ***klock_delare***
Instansnamn: b2v_inst_ klock_delare

6.2.4.1 SYSTEM OCH ARKITEKTUR

Den komponent användas för att dela 50Mhz klockan till 25Mhz för VGA synkpulser och räknare.
KLOCK_DELARE-komponentens symbol visas i Figur 11 och tabell 7 visar In- och utsignalerna.



Figur 11. Klockan 25Mhz för VGA synkpulser och räknare

Tabell 7. In- och utsignalerna till klock_delare

Signal	Namn	Riktning	Typ
Klocksignal 50 MHz	CLOCK_50	in	std_logic
Switch reset	reset_n	in	std_logic
Klocksignal 25 MHz	clk_25	out	std_logic

Ingången signaler
CLOCK_50: 50Mhz input clock.

reset_n: Reset av signalen. Active low.

Utgången signaler
clk_25: 25Mhz input klock för VGA synkpulser och räknare.

7. VERIFIERING

7.1 TESTBÄNKEN

Figur 12 visar teststimuli som bygger på testprotokollet för programmet.

```
   constant sys_clk_period : TIME := 50 ns;
   CLOCK_50       <= NOT CLOCK_50 after 10 ns;  --50 MHz -- varje 20 ns byta värde
   --reset_n_in   <= '0', '1' after 60 ns; -- Först reset_n_in ='0', efter av 50 ns reset_n_in = '1'


test_vga  : PROCESS
 BEGIN


  --- Test case 1 -- Reset_n = 0 och brytaren gör inget
     reset_n <= '0';  -- switchs-reset upp
     key(0) <= '0';   -- brytare(0) trycks inte
     key(1) <= '0';   -- brytare(1) trycks inte
     key(2) <= '0';   -- brytare(2) trycks inte

     WAIT FOR 3*sys_clk_period;

     reset_n <= '1';  -- switchs-reset upp

     WAIT FOR 6*sys_clk_period;

  --- Test case 1 -- Reset_n = 1 och brytare(0) = '1':
  --- Rita en röd fyrkant i övre vänster hörn

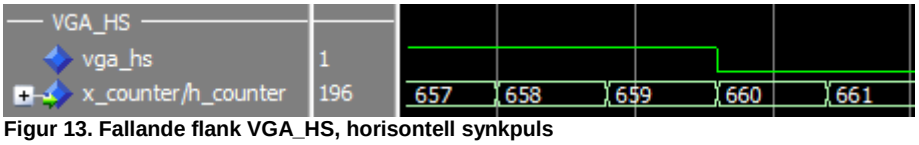
     key <= B"111";            -- Brytare(0) trycks ner
     WAIT;

END PROCESS test_vga ;
END vhdl2_uppgift_1a_arch;
```

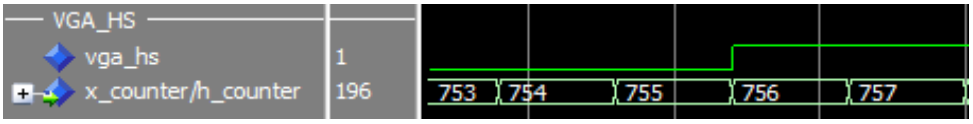
Figur 12. Tesbänken

7.2 BILDER PÅ RESULTATEN FRÅN SIMULERINGEN

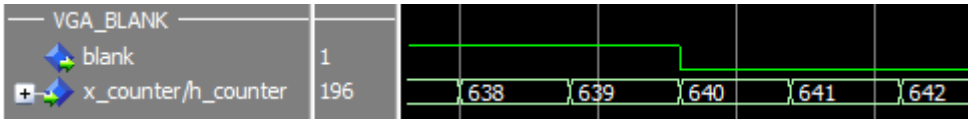
Följande figurer (figur 13-19) visar olika signaler efter verifiering i ModelSim



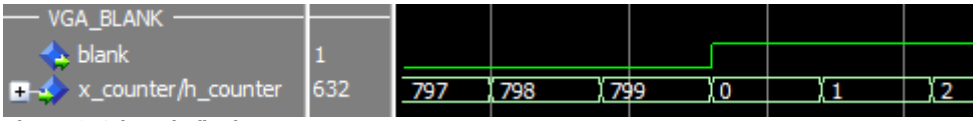
Figur 13. Fallande flank VGA_HS, horisontell synkpuls



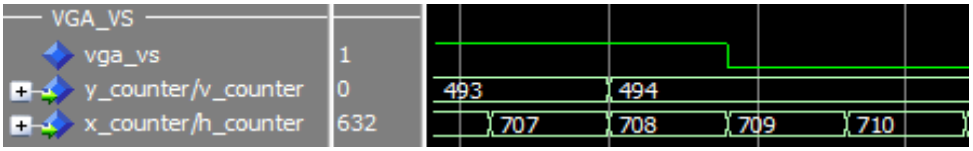
Figur 14. Stigande flank VGA_HS



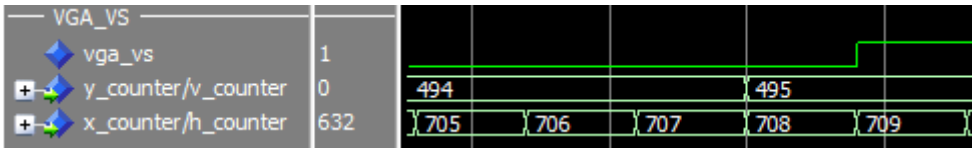
Figur 15. Fallande flank BLANK, räknare utanför synlig bild



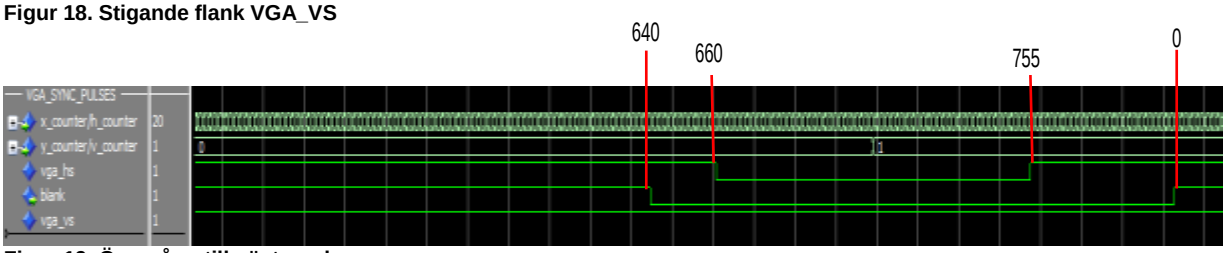
Figur 16. Stigande flank BLANK



Figur 17. Fallande flank VGA_VS, vertikal synkpuls



Figur 18. Stigande flank VGA_VS



Figur 19. Övergång till nästa rad

7.3 IFYLLT TESTPROTOKOLL FÖR VERIFIERING

Tabell 8 visar testprotokoll för verifiering av VGA-kontroller.

Tabell 8. Testprotokoll för verifiering

Testfall	Beskrivning	Acceptansbeskrivning	Verifiering
1	Gör en reset av systemet. Hela skärmen skall rensas vid uppstart av programmet.	Reset_n = '1'	Ja
2	Horisontell sync(vga_hs) med fallande flank.	Horisontell sync(vga_hs) börjar med fallande flank, från horisontell räknaren 660.	Ja
3	Horisontell sync(vga_hs) med stigande flank.	Horisontell sync(vga_hs) slutar med stigande flank, på horisontell räknaren 755	Ja
4	Blank med fallande flank.	Blank(blank) börjar med fallande flank, från horisontell räknaren 640.	Ja
5	Blank med stigande flank.	Blank(blank) slutar med stigande flank, på horisontell räknaren 799.	Ja

6	Vertikal sync(vga_vs) med fallande flank.	Vertikal sync(vga_vs) börjar med fallande flank, från vertikal räknaren 494.	Ja
7	Vertikal sync(vga_vs) med stigande flank.	Vertikal sync(vga_vs) slutar med stigande flank, på vertikal räknaren 494	Ja
8	Övergång till nästa rad	Visa alla signaler till nästa rad	Ja

Följande figurer (figur 7-13) visar olika signaler efter verifiering i ModelSim.

Programmet körs på kortet DE10-Lite är en robust hårdvara design plattform som byggde kring Altera MAX 10 FPGA med en Nios II-CPU som är anpassad för ingenjörsjobbet. DE10-Lite-kortet innehåller en 15-pin D-SUB-kontakt för VGA-utgång, VGA skärm för att visualisering resultaten, VGA-kabel för att skickar signalerna från DE10-Lite kortet till VGA skärm och tre brytarna för att kan testar VGA-kontroller.

8. VALIDERING

Tabell 9 visar testprotokoll för validering och figur 20 visar validering i VGA-skärmen.

Tabell 9. Testprotokoll för validering

Testfall	Beskrivning	Acceptansbeskrivning	Verifiering
1	När trycks ned Sw(0)	Rita en röd fyrkant i övre vänster hörn	Ja
2	När trycks ned Sw(1)	Rita en grön fyrkant i nedre vänster hörn	Ja
3	När trycks ned Sw(2)	Rita en blå fyrkant i nedre höger hörn	Ja
4	När alla switch är nedre	Dessa färger ska överlappa varandra och de ska blandas i mitten av skärmen	Ja



Figur 20. Resultatet i skärmen

9. BILAGA

a. VHDL TOPPNIVÅ

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity vhdl2_uppgift_1a is
	port (
			CLOCK_50 : in std_logic;
			reset_n	: in std_logic;
			key			: in std_logic_vector(2 downto 0);
			vga_hs	: out std_logic;
			vga_vs	: out std_logic;
			vga_r		: out std_logic_vector(3 downto 0);
			vga_g		: out std_logic_vector(3 downto 0);
			vga_b		: out std_logic_vector(3 downto 0)
	);
end vhdl2_uppgift_1a;

architecture rtl of vhdl2_uppgift_1a is

	component klock_delare
			Port (
				CLOCK_50	: in  STD_LOGIC;
				reset_n		: in  STD_LOGIC;
				clk_25mhz : out STD_LOGIC
			);
```

```
	end component;

	component counter
			Port (
				clk_25		: in std_logic;
				reset_n		: in std_logic;
				h_counter: out std_logic_vector(9 downto 0);
				v_counter: out std_logic_vector(9 downto 0);
				blank			: out std_logic;
				vga_hs		: out std_logic;
				vga_vs		: out std_logic

			);
	end component;

	component display_rgb
	port (
			CLOCK_50	: in std_logic;
			reset_n		: in std_logic;
			key			: in std_logic_vector(2 downto 0);
			h_counter: in std_logic_vector(9 downto 0);
			v_counter: in std_logic_vector(9 downto 0);
			vga_r		: out std_logic_vector(3 downto 0);
			vga_g		: out std_logic_vector(3 downto 0);
			vga_b		: out std_logic_vector(3 downto 0)
	);
	end component;


	signal tmp_x_counter, tmp_y_counter	: std_logic_vector (9 downto 0);
	signal tmp_clk_25mhz, tmp_blank		: std_logic;

begin

	b2v_inst_klock_delare : klock_delare
			port map(
			 CLOCK_50	=> CLOCK_50,
			 reset_n	=> reset_n,
			 clk_25mhz => tmp_clk_25mhz
			);

	b2v_inst_counter : counter
			Port map(
```

```
                clk_25          => tmp_clk_25mhz,
                reset_n         => reset_n,
                h_counter=> tmp_x_counter,
                v_counter=> tmp_y_counter,
                blank                   => tmp_blank,
                vga_hs          => vga_hs,
                vga_vs          => vga_vs
        );


    b2v_inst_display_rgb : display_rgb
        Port map(
                CLOCK_50    => CLOCK_50,
                reset_n         => reset_n,
                key                     => key,
                h_counter=> tmp_x_counter,
                v_counter=> tmp_y_counter,
                vga_r           => vga_r,
                vga_g           => vga_g,
                vga_b           => vga_b
        );

end rtl;
```

b. DO-FIL

```
onerror {resume}
quietly WaveActivateNextPane {} 0
add wave -noupdate /vhdl2_uppgift_1a_vhd_tst/CLOCK_50
add wave -noupdate /vhdl2_uppgift_1a_vhd_tst/key
add wave -noupdate /vhdl2_uppgift_1a_vhd_tst/reset_n
add wave -noupdate /vhdl2_uppgift_1a_vhd_tst/vga_b
add wave -noupdate /vhdl2_uppgift_1a_vhd_tst/vga_g
add wave -noupdate /vhdl2_uppgift_1a_vhd_tst/vga_r
add wave -noupdate -divider -height 20 KLOCK_DELARE
add wave -noupdate /vhdl2_uppgift_1a_vhd_tst/i1/b2v_inst_klock_delare/clk_25mhz
add wave -noupdate -divider -height 20 VGA_HS
add wave -noupdate /vhdl2_uppgift_1a_vhd_tst/vga_hs
add wave -noupdate -label x_counter/h_counter -radix unsigned
/vhdl2_uppgift_1a_vhd_tst/i1/b2v_inst_display_rgb/h_counter
add wave -noupdate -divider -height 20 VGA_BLANK
add wave -noupdate /vhdl2_uppgift_1a_vhd_tst/i1/b2v_inst_counter/blank
add wave -noupdate -label x_counter/h_counter -radix unsigned
/vhdl2_uppgift_1a_vhd_tst/i1/b2v_inst_display_rgb/h_counter
```

```
add wave -noupdate -divider -height 20 VGA_VS
add wave -noupdate /vhdl2_uppgift_1a_vhd_tst/vga_vs
add wave -noupdate -label y_counter/v_counter -radix unsigned
/vhdl2_uppgift_1a_vhd_tst/i1/b2v_inst_display_rgb/v_counter
add wave -noupdate -label x_counter/h_counter -radix unsigned
/vhdl2_uppgift_1a_vhd_tst/i1/b2v_inst_display_rgb/h_counter
add wave -noupdate -divider -height 20 VGA_SYNC_PULSES
add wave -noupdate -label x_counter/h_counter -radix unsigned
/vhdl2_uppgift_1a_vhd_tst/i1/b2v_inst_display_rgb/h_counter
add wave -noupdate -label y_counter/v_counter -radix unsigned
/vhdl2_uppgift_1a_vhd_tst/i1/b2v_inst_display_rgb/v_counter
add wave -noupdate /vhdl2_uppgift_1a_vhd_tst/vga_hs
add wave -noupdate /vhdl2_uppgift_1a_vhd_tst/i1/b2v_inst_counter/blank
add wave -noupdate /vhdl2_uppgift_1a_vhd_tst/vga_vs
TreeUpdate [SetDefaultTree]
WaveRestoreCursors {{Cursor 1} {0 ps} 0}
quietly wave cursor active 0
configure wave -namecolwidth 152
configure wave -valuecolwidth 40
configure wave -justifyvalue left
configure wave -signalnamewidth 1
configure wave -snapdistance 10
configure wave -datasetprefix 0
configure wave -rowmargin 4
configure wave -childrowmargin 2
configure wave -gridoffset 0
configure wave -gridperiod 1
configure wave -griddelta 40
configure wave -timeline 0
configure wave -timelineunits ns
update
WaveRestoreZoom {26293989 ps} {27112853 ps}
```

c. TESTBÄNK

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY vhdl2_uppgift_1a_vhd_tst IS
END vhdl2_uppgift_1a_vhd_tst;
ARCHITECTURE vhdl2_uppgift_1a_arch OF vhdl2_uppgift_1a_vhd_tst IS
-- constants
-- signals
SIGNAL CLOCK_50 : STD_LOGIC := '0';
```

```
SIGNAL key : STD_LOGIC_VECTOR(2 DOWNTO 0);
SIGNAL reset_n : STD_LOGIC;
SIGNAL vga_b : STD_LOGIC_VECTOR(3 DOWNTO 0);
SIGNAL vga_g : STD_LOGIC_VECTOR(3 DOWNTO 0);
SIGNAL vga_hs : STD_LOGIC;
SIGNAL vga_r : STD_LOGIC_VECTOR(3 DOWNTO 0);
SIGNAL vga_vs : STD_LOGIC;


constant sys_clk_period : TIME := 50 ns;


COMPONENT vhdl2_uppgift_1a
	PORT (
	CLOCK_50 : IN STD_LOGIC;
	key : IN STD_LOGIC_VECTOR(2 DOWNTO 0);
	reset_n : IN STD_LOGIC;
	vga_b : BUFFER STD_LOGIC_VECTOR(3 DOWNTO 0);
	vga_g : BUFFER STD_LOGIC_VECTOR(3 DOWNTO 0);
	vga_hs : BUFFER STD_LOGIC;
	vga_r : BUFFER STD_LOGIC_VECTOR(3 DOWNTO 0);
	vga_vs : BUFFER STD_LOGIC
	);
END COMPONENT;
BEGIN
	i1 : vhdl2_uppgift_1a
	PORT MAP (
-- list connections between master ports and signals
	CLOCK_50 => CLOCK_50,
	key => key,
	reset_n => reset_n,
	vga_b => vga_b,
	vga_g => vga_g,
	vga_hs => vga_hs,
	vga_r => vga_r,
	vga_vs => vga_vs
	);


	CLOCK_50 			<= NOT CLOCK_50 after 10 ns;		--50 MHz --
varje 20 ns byta värde
	--reset_n_in   <= '0', '1' after 60 ns; -- Först reset_n_in ='0', efter av 50 ns
reset_n_in = '1'
```

```
test_vga  : PROCESS
BEGIN


 --- Test case 1 -- Reset_n = 0 och brytaren gör inget
        reset_n        <= '0';          -- switchs-reset upp
        key(0) <= '0';                  -- brytare(0) trycks inte
        key(1) <= '0';                  -- brytare(1) trycks inte
        key(2) <= '0';                  -- brytare(2) trycks inte

        WAIT FOR 3*sys_clk_period;

        reset_n        <= '1';          -- switchs-reset upp

        WAIT FOR 6*sys_clk_period;

 --- Test case 1 -- Reset_n = 1 och brytare(0) = '1':
 --- Rita en röd fyrkant i övre vänster hörn

        key <= B"111";                                                  -- Brytare(0)
trycks ner
        WAIT;

END PROCESS test_vga ;
END vhdl2_uppgift_1a_arch;
```